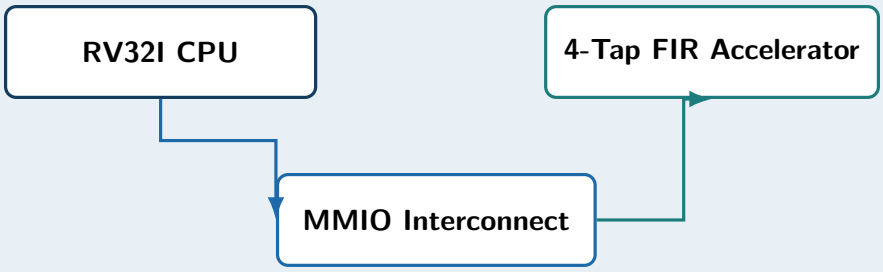
RV32I RISC-V SoC

with Custom FIR DSP Accelerator

Technical Project Report

Prepared by Bekuretsion Tadios



Implementation:	SystemVerilog / Verilator / GTKWave
Software flow:	GNU RISC-V toolchain, RV32I / ILP32
Repository:	https://github.com/bekuretsion/riscv-dsp-soc
Status:	Functional SoC + benchmark + waveform verification

Contents

1 Executive Summary

This project implements a compact RISC-V system-on-chip in SystemVerilog and integrates a custom 4-tap finite impulse response (FIR) DSP accelerator as a memory-mapped peripheral. The processor executes RV32I software generated by the GNU RISC-V toolchain, performs normal RAM accesses, and controls the accelerator using ordinary `LW` and `SW` instructions through a simple MMIO interconnect.

The design was developed and verified incrementally: ALU, register file, decoder, datapath, instruction fetch, branches, jumps, load/store, the FIR accelerator, MMIO routing, and finally CPU-driven accelerator execution. A scaling benchmark over 8, 16, and 32 samples showed repeatable hardware acceleration of approximately **2.5**× compared with the software FIR implementation used in the project.

Main result: software and hardware FIR outputs match, while the hardware path achieves about 2.5× lower cycle count in the scaling benchmark.

2 Project Objectives

The project was designed to demonstrate practical hardware/software co-design rather than only an isolated CPU or isolated DSP block. The key objectives were:

- Build a working 32-bit RISC-V processor from RTL fundamentals.
- Execute real machine code generated by the GNU RISC-V toolchain.
- Support arithmetic, load/store, branch, jump, and address-generation operations required by the test software.
- Implement and verify a programmable 4-tap FIR accelerator.
- Expose the accelerator through memory-mapped I/O.
- Allow CPU software to configure coefficients, provide samples, trigger computation, and read results.
- Verify functionality at unit, subsystem, and full-SoC levels.
- Measure software-versus-hardware cycle counts for the FIR workload.

3 SoC Architecture

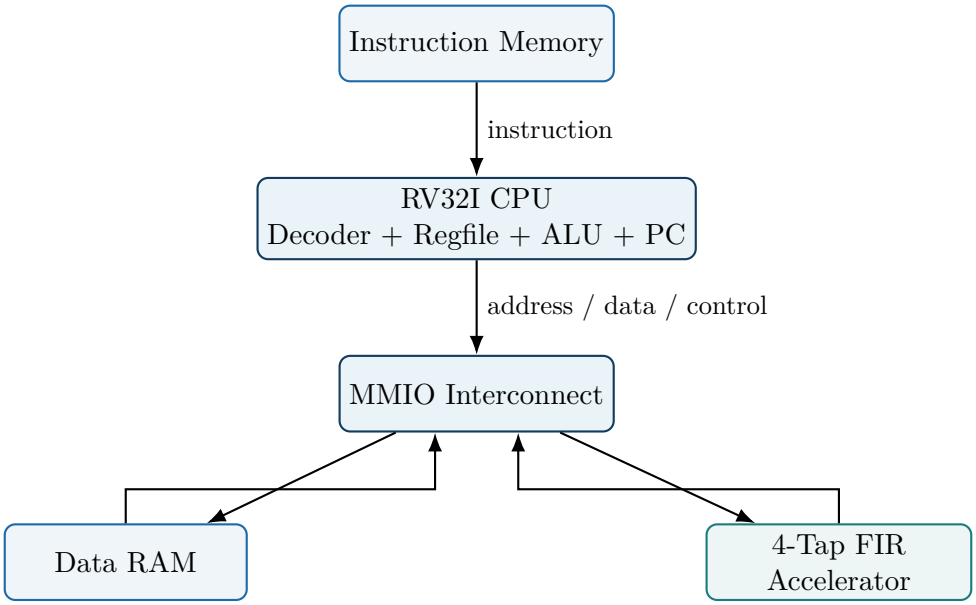


Figure 1: High-level architecture of the implemented RISC-V DSP SoC.

The CPU sees a single address space. Normal low addresses are routed to data RAM, while addresses in the `0x4000_xxxx` region are routed to the FIR accelerator. This allows accelerator access using standard load/store instructions with no custom ISA extension.

4 RV32I CPU Design

4.1 Datapath

The CPU datapath contains a 32-bit ALU, a 32-register integer register file, immediate generation, an ALU operand multiplexer, writeback selection, and the memory/MMIO path. The design uses a single-cycle style for the implemented educational core: each instruction is fetched and its effects are committed on the active clock edge.

4.2 Instruction Support

The current implementation includes the instructions required by the included software and tests.

Table 1: Implemented and verified instruction subset.

Class	Instructions	Role
Arithmetic	ADD, SUB, ADDI	Integer arithmetic and address/sample generation
Logic	AND, OR, XOR	Basic RV32I logical operations
Shift	SLL, SRL, SRA	Shift operations
Compare	SLT, SLTU	Signed and unsigned comparisons
Memory	LW, SW	RAM and MMIO access
Branch	BEQ, BNE	Conditional control flow and loops
Jump	JAL	Unconditional jump / terminal loop
Upper immediate	LUI	Builds addresses such as `0x40000000`

4.3 Control Flow

Branches are implemented by subtracting the two source operands in the ALU and using the zero flag. For `BEQ`, a zero result takes the branch; for `BNE`, a non-zero result takes the branch. Branch and jump targets are formed by adding the decoded immediate to the current PC.

4.4 Program Loading

Assembly software is compiled with the GNU RISC-V toolchain for `rv32i/ilp32`. The ELF file is converted to a raw binary, then a Python helper converts little-endian 32-bit words into a `.hex` file loaded using `$readmemh`.

Listing 1: Software build flow.

```
riscv64-unknown-elf-gcc -march=rv32i -mabi=ilp32 \
  -nostdlib -nostartfiles -Ttext=0x0 software/fir_test.S \
  -o software/fir_test.elf

riscv64-unknown-elf-objcopy -O binary \
  software/fir_test.elf software/fir_test.bin

python3 scripts/bin2hex.py software/fir_test.bin programs/fir_test.hex
```

5 FIR DSP Accelerator

The accelerator implements a programmable four-tap finite impulse response filter. For sample $x[n]$ and coefficients $c_0 \dots c_3$, the output is

$$y[n] = c_0 x[n] + c_1 x[n-1] + c_2 x[n-2] + c_3 x[n-3]. \tag{1}$$

The hardware keeps three previous samples in an internal delay line and evaluates four signed products plus an accumulator. On a write of `1` to the control register, the result register is updated, the delay line shifts, and `done` pulses for one cycle.

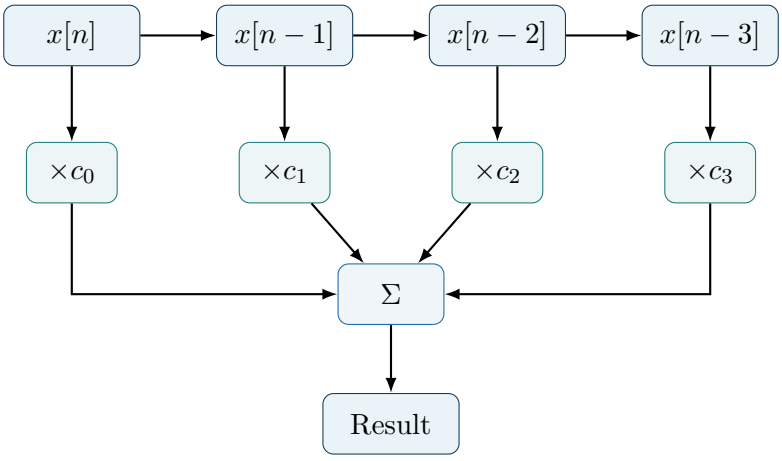


Figure 2: Conceptual 4-tap FIR datapath.

6 Memory-Mapped I/O

The FIR accelerator is exposed at base address `0x40000000`. The interconnect selects the FIR peripheral when the upper address bits match the configured MMIO region; otherwise accesses go to normal RAM.

Table 2: FIR memory-mapped register map.

Address	Register	Description
0x40000000	**INPUT**	Input sample register
0x40000004	**COEFF0**	Coefficient c_0
0x40000008	**COEFF1**	Coefficient c_1
0x4000000C	**COEFF2**	Coefficient c_2
0x40000010	**COEFF3**	Coefficient c_3
0x40000014	**CONTROL**	Write bit 0 = 1 to execute one sample
0x40000018	**RESULT**	Last FIR result
0x4000001C	**STATUS**	Bit 0 reflects `done`

6.1 Software-Controlled FIR Example

The CPU uses `LUI` to construct the accelerator base address and then standard stores/loads to operate it.

Listing 2: Representative CPU-to-FIR MMIO sequence.

```
lui x10, 0x40000 # x10 = 0x40000000

addi x5, x0, 1
sw x5, 4(x10) # c0 = 1

addi x5, x0, 10
sw x5, 0(x10) # input = 10

addi x5, x0, 1
sw x5, 20(x10) # start

lw x9, 24(x10) # x9 <- result
```

The integrated SoC test verified that `x10` became `0x40000000`, coefficient writes reached the accelerator, the input and control stores were accepted, and the result was read back into CPU register `x9` as 10.

7 Verification Strategy

Verification was built incrementally so integration failures could be localized quickly.

Table 3: Verification layers used in the project.

Level	Purpose	Result
ALU unit test	Arithmetic, logical, shifts, compares	PASS
Register file	Read/write behavior and x0 semantics	PASS
Decoder	Instruction fields, immediates, control signals	PASS
Datapath	Register/ALU/writeback integration	PASS
CPU core	Real ADDI/ADD instruction execution	PASS
Fetch loop	PC + instruction-memory execution	PASS
Load/store	RAM writes and reads through LW/SW	PASS
Branches	BEQ/BNE target and loop execution	PASS
Jump	JAL terminal loop behavior	PASS
FIR unit test	Coefficients, delay line, MAC, result	PASS
MMIO test	RAM/FIR address routing and isolation	PASS
Full SoC	CPU-controlled FIR through MMIO	PASS
Benchmarks	Software/hardware result equivalence and cycles	PASS

8 Waveform Evidence

Figure 3 shows a GTKWave trace from the integrated SoC. The PC and instruction streams progress as expected, while `fir_selected` asserts during accesses to the FIR MMIO region. The trace provides timing-level evidence of software-driven peripheral interaction.

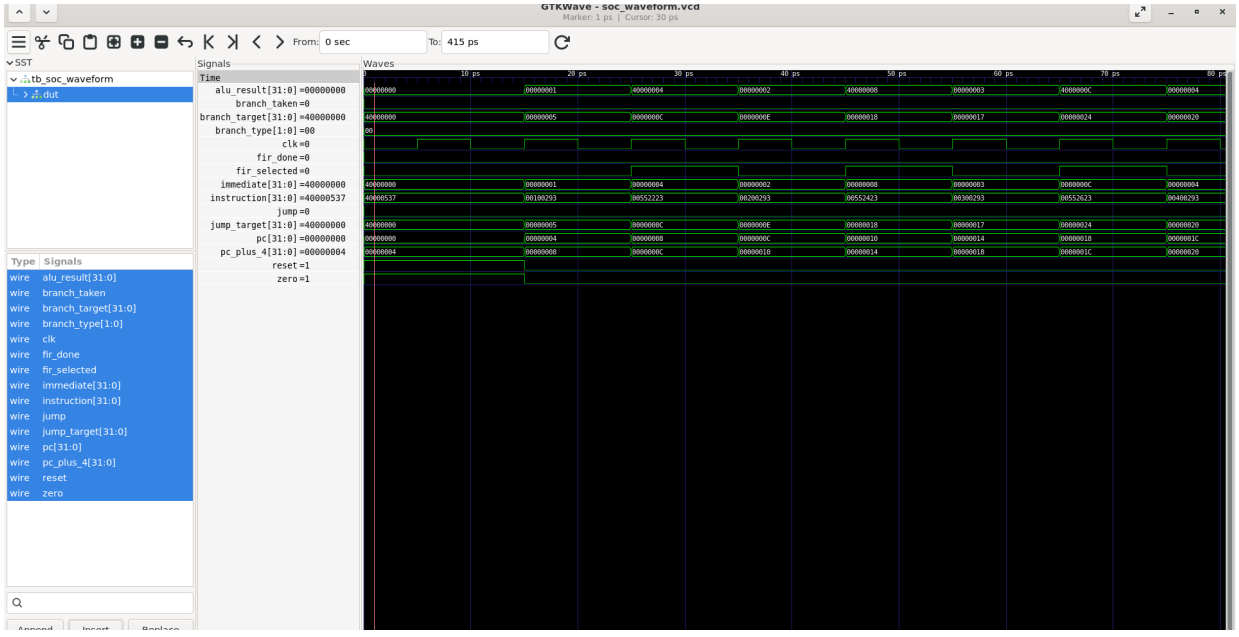


Figure 3: GTKWave capture of the integrated CPU/MMIO/FIR simulation.

9 Performance Evaluation

Two forms of comparison were performed during development. An initial 8-sample test showed a 3.22× reduction in cycle count for its specialized sequence. A more systematic scaling benchmark then processed 8, 16, and 32 samples and produced the consistent results below.

Table 4: FIR scaling benchmark.

Samples	Software cycles	Hardware cycles	Speedup
8	119	47	2.53×
16	239	95	2.52×
32	479	191	2.51×

At 32 samples, the average cycle cost is

$$\text{software} = \frac{479}{32} \approx 14.97 \text{ cycles/sample}, \tag{2}$$

$$\text{hardware} = \frac{191}{32} \approx 5.97 \text{ cycles/sample}. \tag{3}$$

Therefore,

$$\text{speedup} = \frac{479}{191} \approx 2.51 \times . \tag{4}$$

The near-constant speedup from 8 to 32 samples indicates a repeatable throughput advantage in this benchmark rather than a one-off result caused only by fixed setup cost.

10 Reproducible Build and Test Flow

The Makefile exposes separate regression targets so unit tests and benchmark programs do not overwrite each other's intent.

Listing 3: Representative project commands.

```
make clean
make soc-test # CPU -> FIR integrated regression
make fir # standalone FIR verification
make mmio # interconnect verification
make fair-benchmark # fair 8-sample benchmark
make scaling-benchmark # 8/16/32-sample scaling benchmark
```

Waveform generation uses Verilator tracing and GTKWave:

```
verilator --binary --timing --trace ... tb/tb_soc_waveform.sv \
  --top-module tb_soc_waveform
./obj_dir/Vtb_soc_waveform
gtkwave soc_waveform.vcd
```

11 Engineering Notes and Limitations

The project is intentionally compact and educational rather than a complete production RISC-V implementation. Important limitations include:

- The CPU implements the instruction subset required by the current tests, not the entire RV32I specification.
- The core is not yet a five-stage pipelined implementation with hazard forwarding/stalling.
- The FIR accelerator uses a simple register-oriented MMIO interface rather than DMA or streaming FIFOs.

- Accelerator completion is exposed as a status signal/register rather than an interrupt.
- The FIR output currently truncates the 64-bit accumulator to 32 bits.
- The current performance numbers are simulation cycle measurements for this RTL/software pair; FPGA timing and resource utilization still require synthesis.
- Several Verilator warnings are currently benign (for example unused upper address bits and final-newline warnings) and can be cleaned during repository polish.

These constraints are useful next-step opportunities rather than hidden behavior; they define a clear path from the current proof-of-concept SoC to a more production-like architecture.

12 Recommended Next Steps

A logical extension roadmap is:

1. Clean lint/newline warnings and maintain a zero-error regression suite.
2. Add broader RV32I coverage, especially `JALR` and additional immediate operations needed for easier C support.
3. Add a hardware cycle counter/performance CSR or MMIO timer.
4. Synthesize on an FPGA target and report LUT, FF, DSP, BRAM, Fmax, and timing slack.
5. Pipeline the FIR accelerator or add a streaming FIFO interface.
6. Upgrade the CPU to a multi-stage pipeline with hazard detection and forwarding.
7. Compare software FIR, MMIO FIR, and a streaming/DMA FIR architecture over larger datasets.

13 Conclusion

The project demonstrates a complete hardware/software co-design path: RISC-V assembly is built with the GNU toolchain, executed by a custom SystemVerilog RV32I CPU, routed through a memory-mapped interconnect, processed by a programmable FIR hardware accelerator, and verified by both functional assertions and waveform inspection.

The key technical achievement is not only that the FIR arithmetic works, but that real software running on the custom CPU controls the accelerator using standard RISC-V load/store semantics. The measured 8/16/32-sample scaling benchmark reports a repeatable speedup of approximately **2.5**× for the implemented FIR workload, while software and hardware results remain identical at the measured checkpoints.

Status: working RV32I SoC + MMIO FIR accelerator + toolchain integration + waveform evidence + repeatable performance benchmark.